



# Transformation & Action

## Part 1

[www.3aayaam.in](http://www.3aayaam.in)

# Agenda

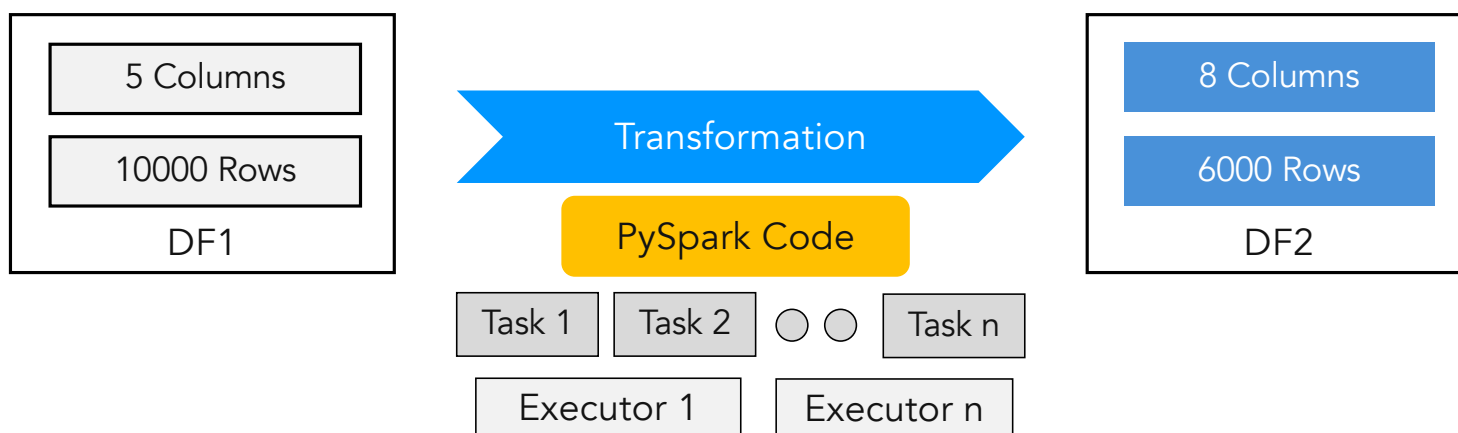
1. Basic Transformation & Action
2. Lazy Evaluation
3. Actions and Jobs (Spark UI)
4. “load” – Transformation or Action
5. Special Transformations
6. Spark Shuffle
7. Application, Job, Stage
8. Spark Architecture & Application

# Basic Transformation & Action

Pre-requisite: SQL Essentials

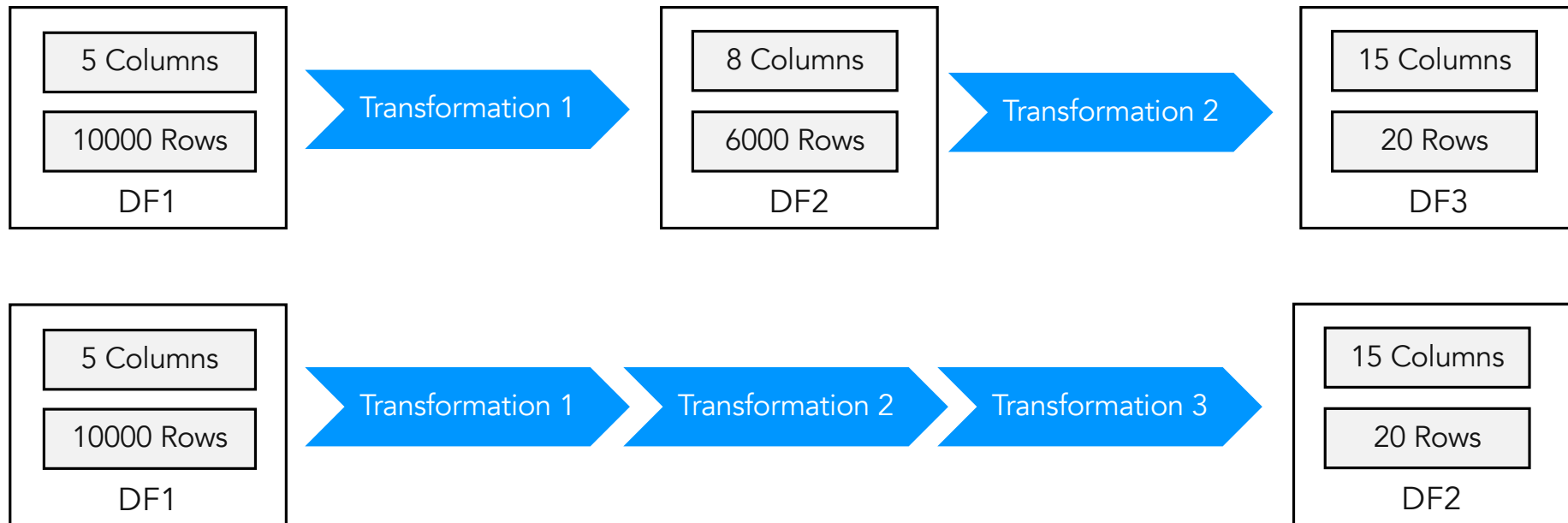
# Transformation

- Change the structure and data of a DataFrame and create a new DataFrame out of it.



- Transformations are always applied on DataFrames.
- Transformation creates a new DF with new form & shape from an existing DF.
- The DataFrames are not created immediately. Lazy Evaluation.

# Transformation



# Transformation

## Transformations

- `df.columns`
- `df.distinct()`
- `df.filter()`
- `df.where()`
- `df.limit()`
- `df.printSchema()`
- `df.select()`

`df.show()` – Will bring the records in driver node

```
SHOW COLUMNS
```

```
SELECT DISTINCT(col) FROM table
```

```
SELECT * FROM table WHERE col = value
```

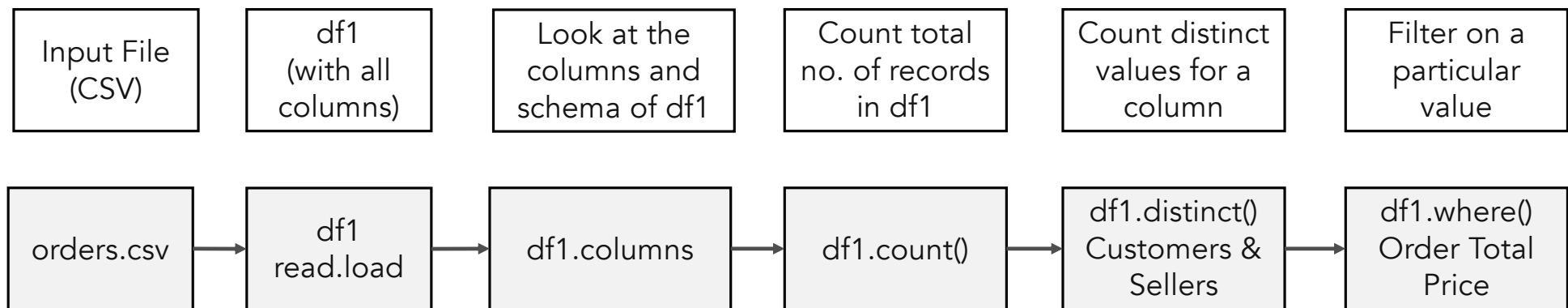
```
SELECT * FROM table LIMIT 10
```

```
SHOW COLUMNS
```

```
SELECT col1, col2, col3 FROM table
```

Let's see them in action

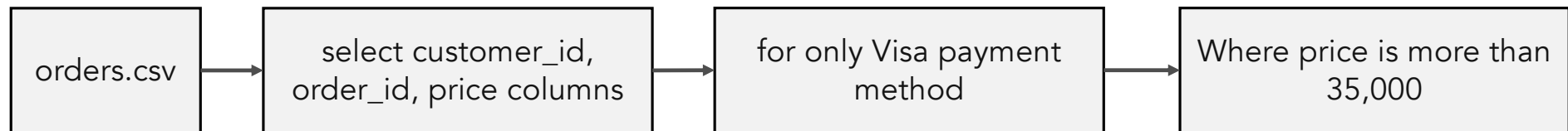
# Transformation



- Transform one-by-one.

Let's see it in action

# Transformation



- One-by-one transformation.
- Chain transformation.

Let's see it in action



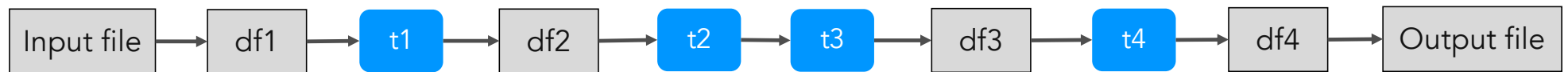
# Transformation

- `df.describe(['col1', 'col2'])` – shows basic statistics for exploratory purposes.
- `df.limit(n)` – limits the count of output rows.
- `df.sort("col")` – creates a new DataFrame that is sorted by specified column. There can be one or more columns.
- `df.drop("col")` – creates a new DataFrame after removing the mentioned columns.
- `df.dropDuplicates(['col1', 'col2'])` – creates a new DataFrame after removing duplicate rows.
- `df.dropna()` – creates a new DataFrame after removing rows with NULL values.

Let's HandsOn

# Action

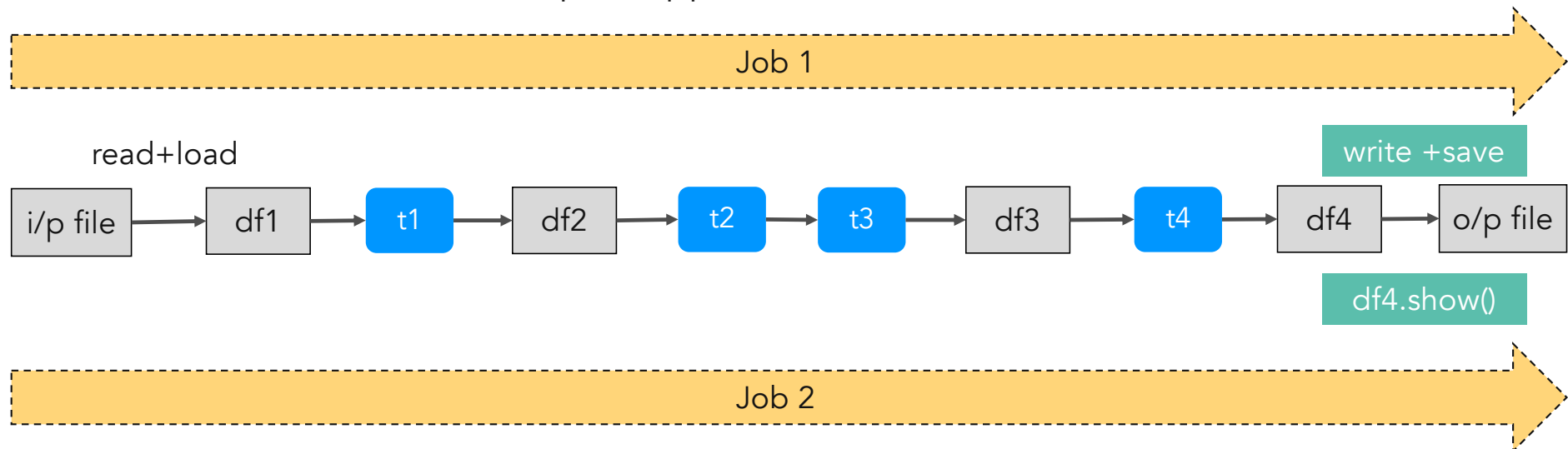
- Action starts DAG processing – create DataFrames by applying transformations.



- Action triggers DAG.
- One action creates one Spark job.
- **`df.show()`**
- **`df.count()`**
- **`df.write.mode(...).save(...)`**
- **`df.first()`** – returns the first row.
- **`df.head(n)`** – returns first n rows.
- **`df.tail(n)`** – returns last n rows.

# Lazy Evaluation

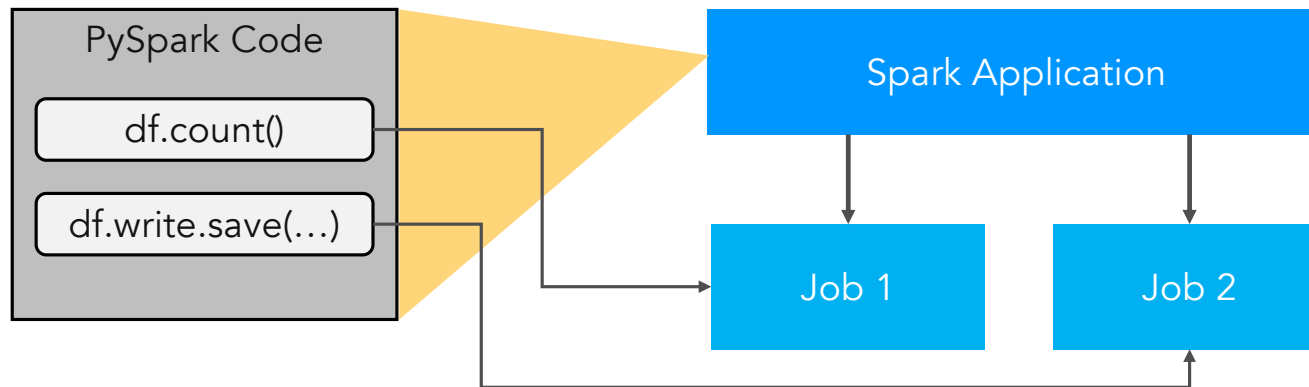
- DAG processing ??
- Actions trigger DAGs. What is trigger ??
- An action will start the processing from beginning.
- Action will start a job for a Spark application.



# Action & Job

- Each action corresponds to a job in Spark.
- Hierarchy: Spark Application → Jobs → ...

Let's HandsOn



# Application & Spark UI

- Application, Job.
- Environment variables.
- Executor details

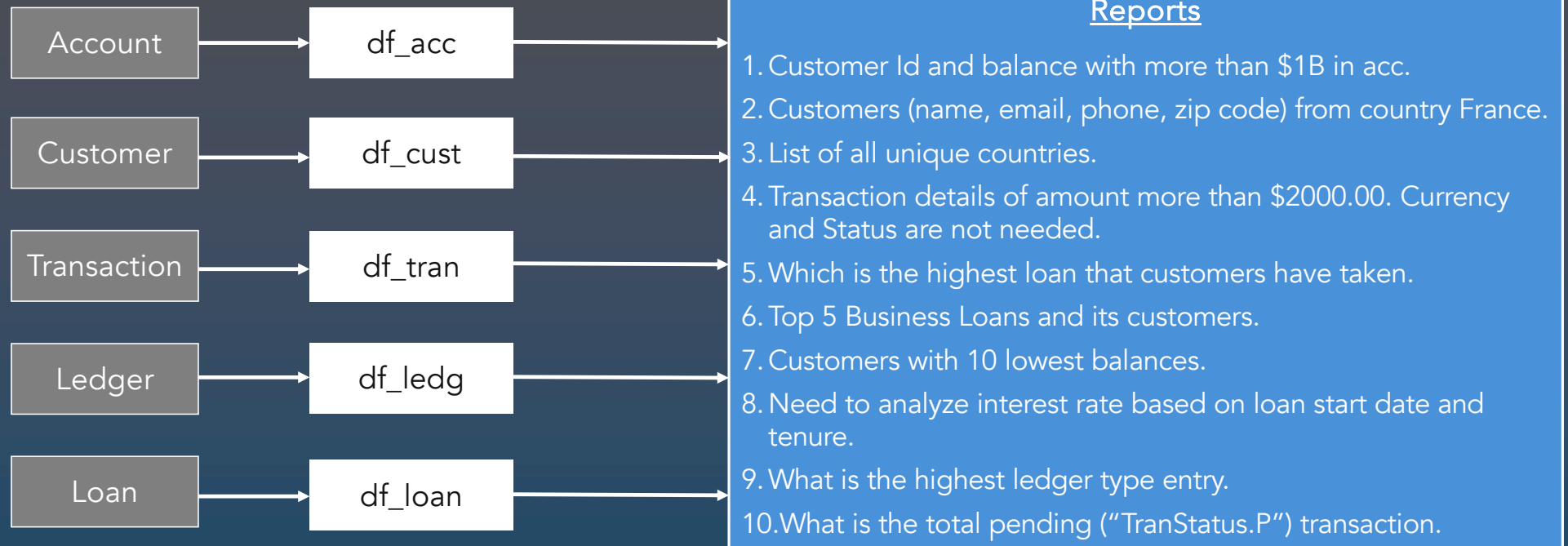
# Is “load” a Transformation or Action

| Operation (with <code>df.show()</code> )     | Job Created? | Action             |
|----------------------------------------------|--------------|--------------------|
| Load CSV without header and inferSchema      | Yes          | Load & Show        |
| Load CSV with header and without inferSchema | Yes          | Load & Show        |
| Load CSV with header and inferSchema         | Yes          | Load, Infer & Show |
| Load JSON                                    | Yes          | Load & Show        |
| Load Parquet                                 | Yes          | Load & Show        |
| Load Avro                                    | Yes          | Show               |

# Performance Comparison

| Operation                                | Src: csv<br>Tgt: csv | Src: csv<br>Tgt: parq | Src: parq<br>Tgt: parq | Src: parq<br>Tgt: csv |
|------------------------------------------|----------------------|-----------------------|------------------------|-----------------------|
| 1. Read → Write                          |                      |                       |                        |                       |
| 2. Read → Select → Filter → Sort → Write |                      |                       |                        |                       |
| 3. Read → Select → Distinct → Show       |                      |                       |                        |                       |
| 4. Read → Sort → Select → Show           |                      |                       |                        |                       |
| 5. Read → Select → Sort → Drop → Show    |                      |                       |                        |                       |
| 6. Read → Count → Show                   |                      |                       |                        |                       |

# HandsOn – Project 1 (Finance)



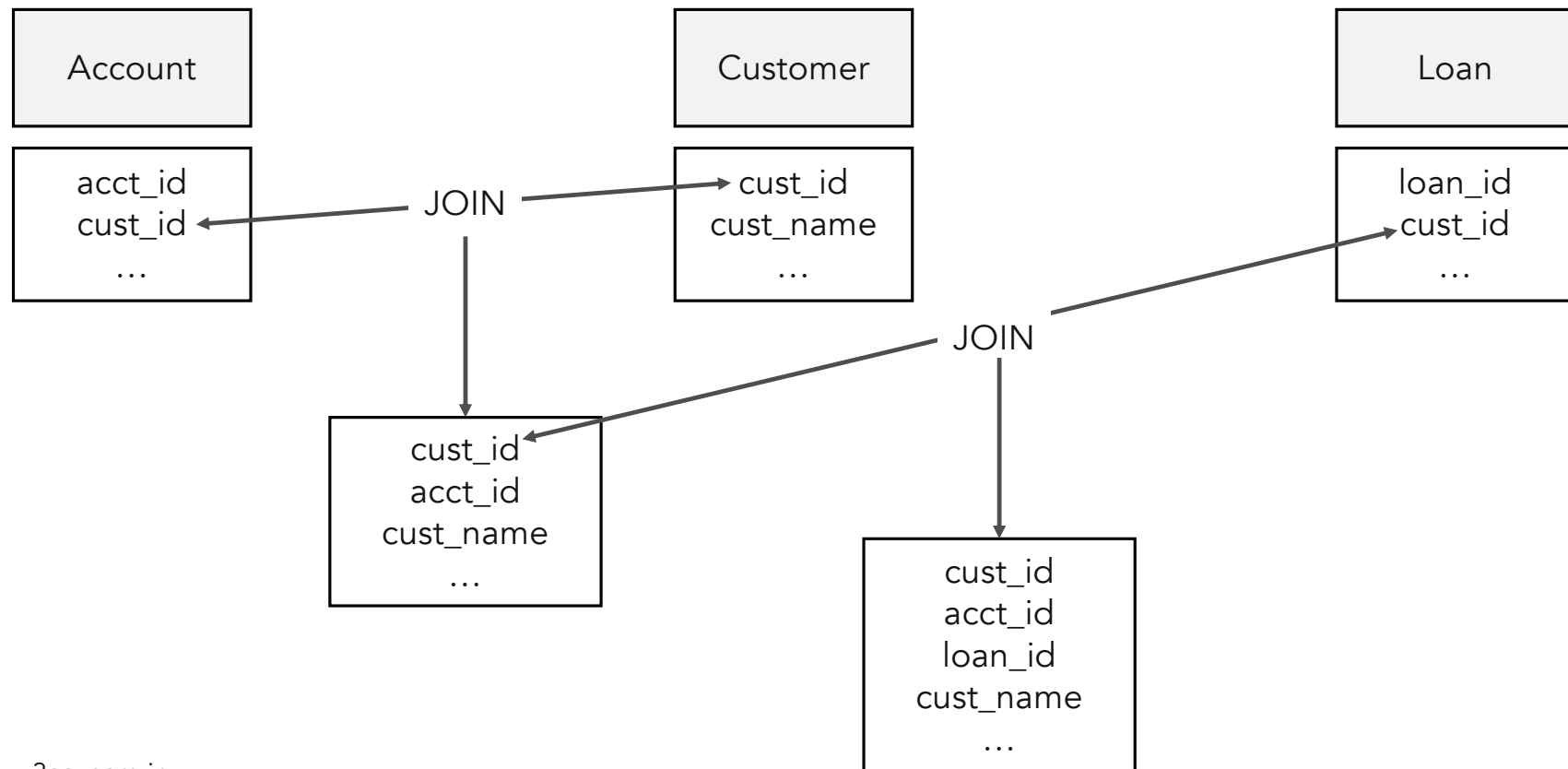


# More Spark Transformations

# Transformations

- `agg({"<col-name>" : "<aggr-function>"}). min, max, avg, count, sum`
- `df.groupBy("<col-name>").agg()/max()/min()/sum()/avg()`
- `df.select(df.<col-name>)`
- `df1.join(df2, '<col1-name>', 'join-type')`
- `df.orderBy("<one-or-more-columns>", desc/asc("<some-col>"))`
- `df.selectExpr("<col-with-runtime-functions>")`
- `df1.subtract(df2)` → Rows which are there df1, but not in df2
- `df1.union(df2), df1.unionAll(df2)`
- `df.withColumn()` → Add a new column in the DataFrame. (`df.withColumns()`)

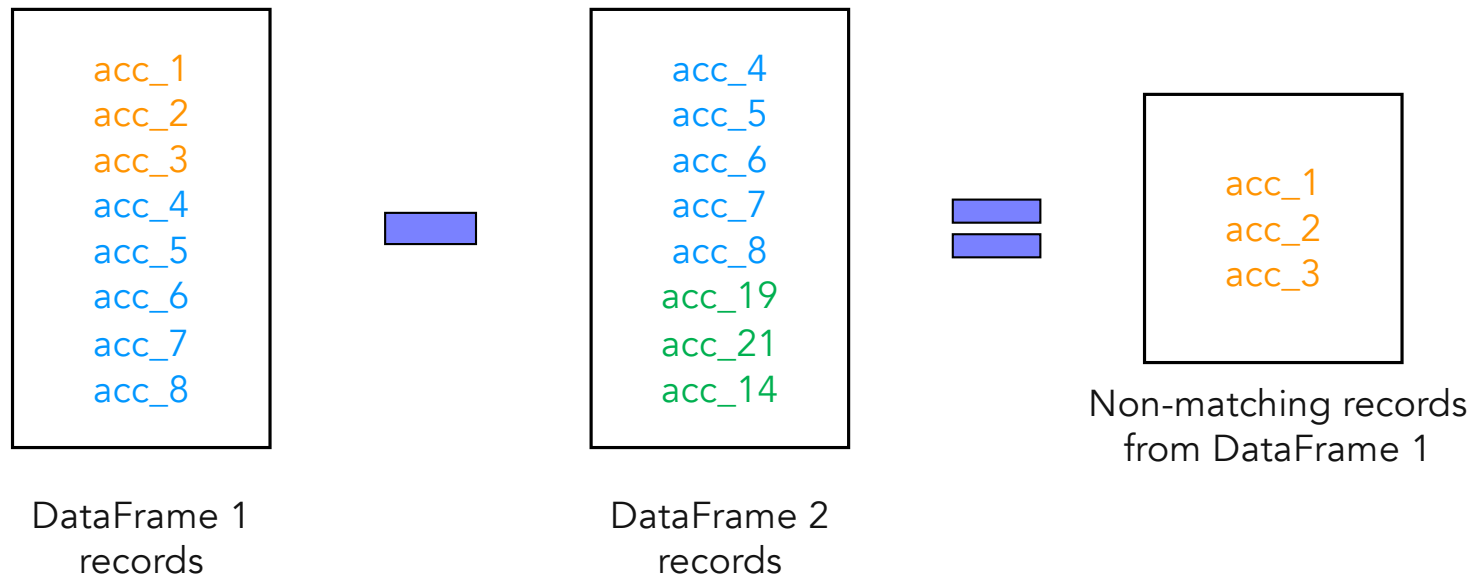
# Join



# "col" function

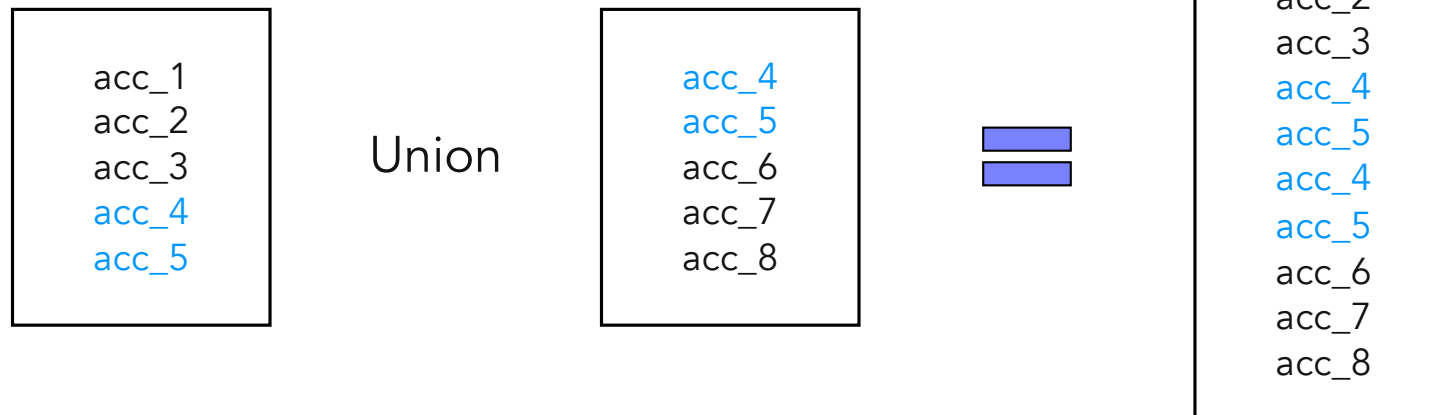
- `from pyspark.sql.functions import col`
- Why use col function?
- Methods
  - `alias()`
  - `asc()` - with `orderBy`
  - `desc()` - with `orderBy`
  - `between(<start>, <end>)` - with filter
  - `isNull, isNotNull`
  - `cast(<data-type>)` - with select
  - `contains(<string>)` - with filter
- Methods
  - `startswith(<string>)` - with filter
  - `endswith(<string>)` - with filter
  - `like(<string>)` - with filter
  - `substr(<pos>, <length>)` - with select
  - `dropFields()` - with select
- Some of the functions can be used with `<dataFrame>.<columnName>` (`df_cust.cust_name`)

# Subtract



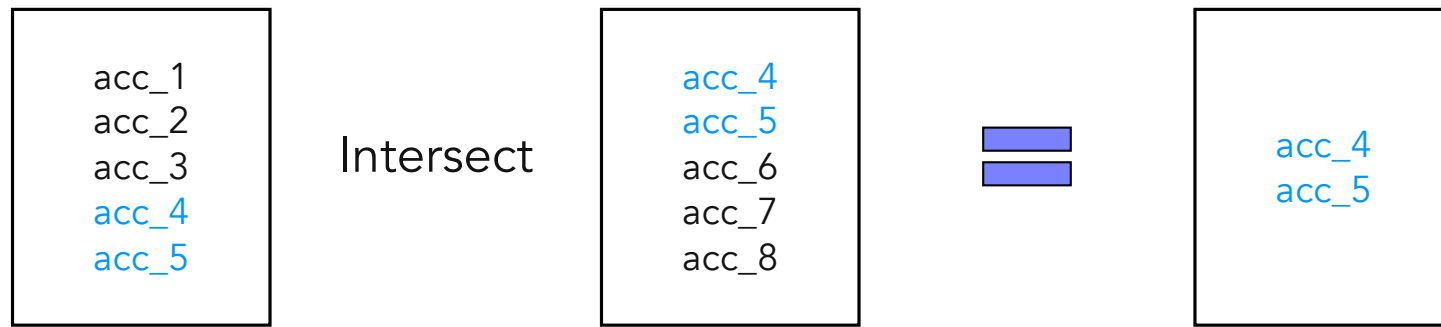
- Cannot be done on individual columns. Entire row is considered.
- Select transformation can be used to select same columns from all dataframes.

# Union & UnionAll



- Duplicate records will not be removed.
- The no. of columns in both the dataframes should be same.
- Cannot be done on individual columns. Entire row is considered.
- Select transformation can be used to select same columns from all dataframes.

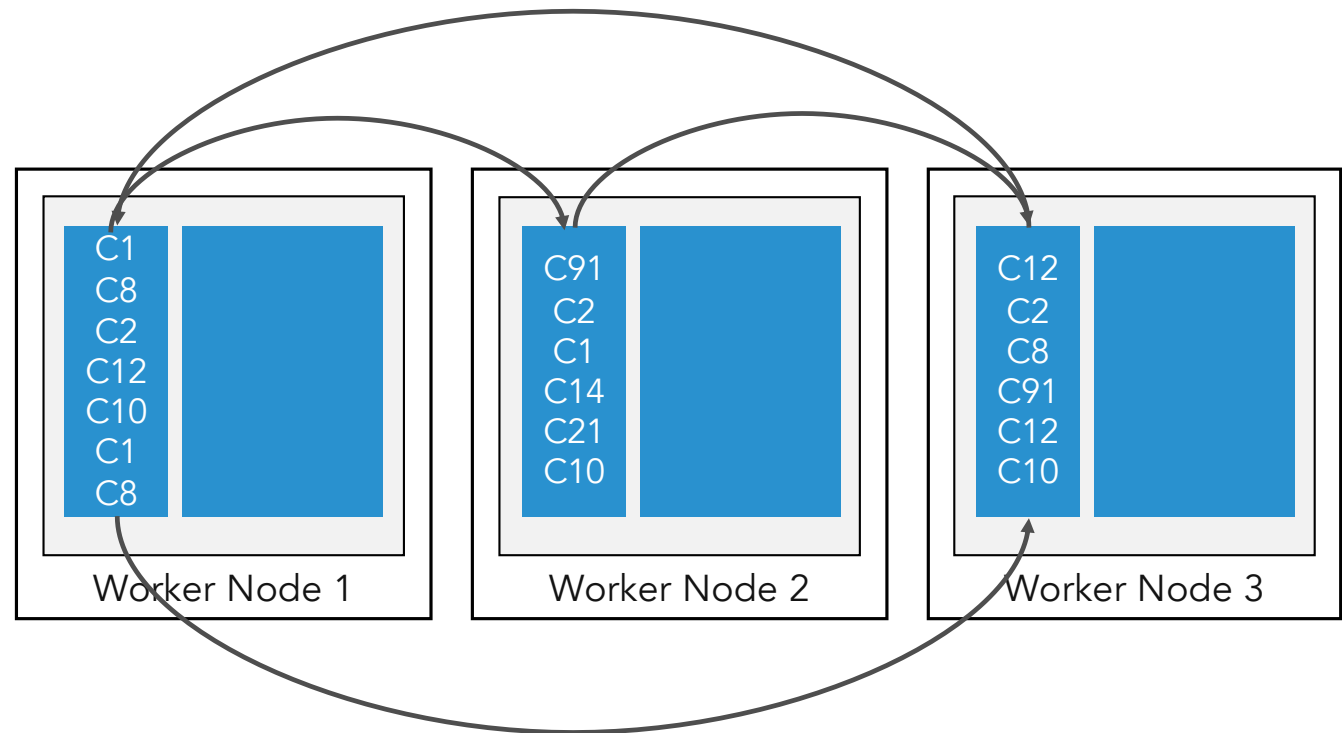
# Intersect



- Returns common rows.
- The no. of columns in both the dataframes should be same.
- Cannot be done on individual columns.
- Select transformation can be used to select same columns from all dataframes.

# Special Transformations

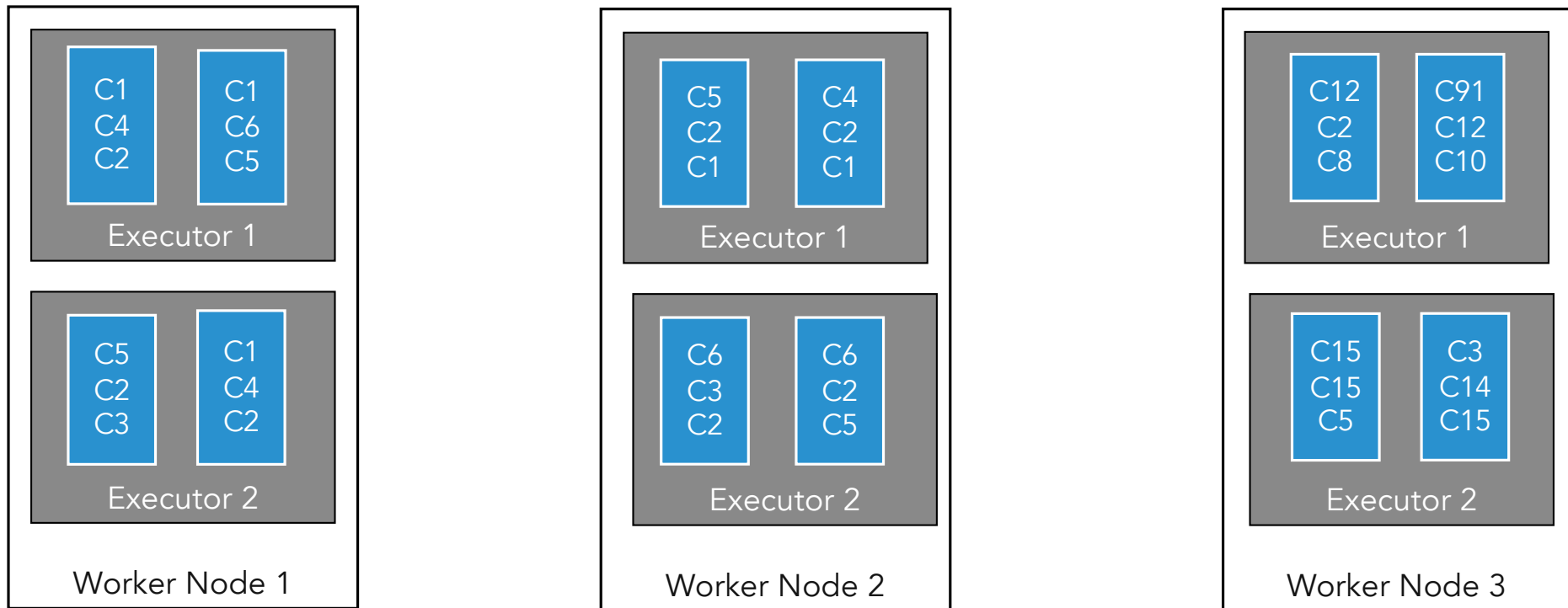
- `df.groupBy`
- `df.orderBy`
- `df.sort`
- `df.distinct`
- `df.dropDuplicates`
- `df.join`
- **Wide Transformation**
- **Narrow Transformation**





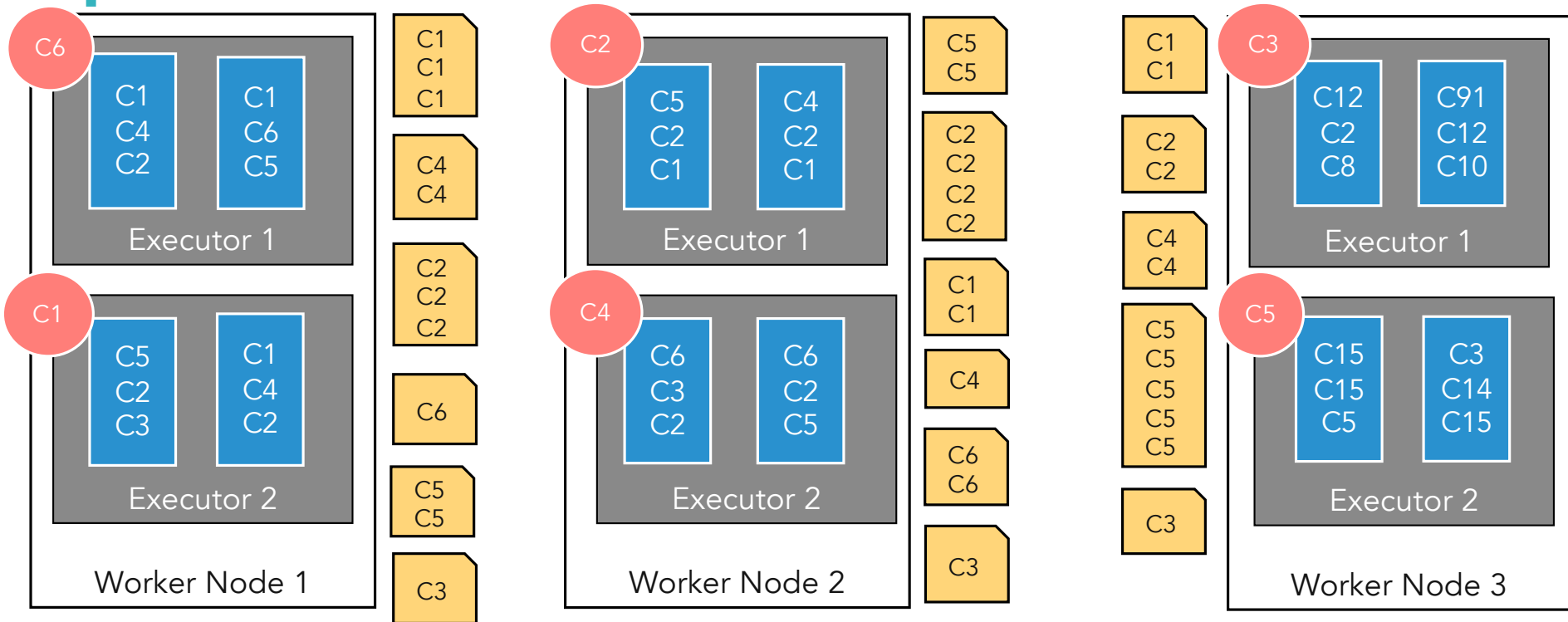
# Shuffling Transformations

# Spark Shuffle



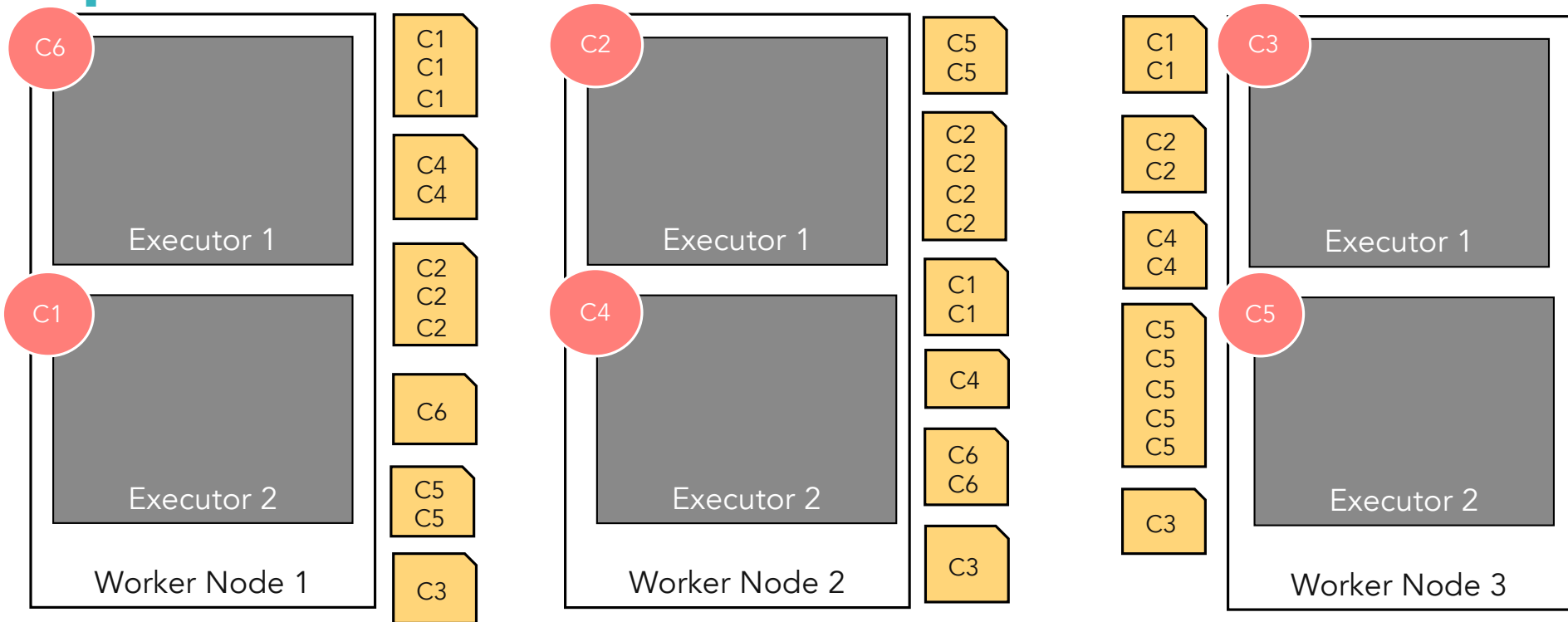
C1, C2, C3, C4, C5, C6

# Spark Shuffle



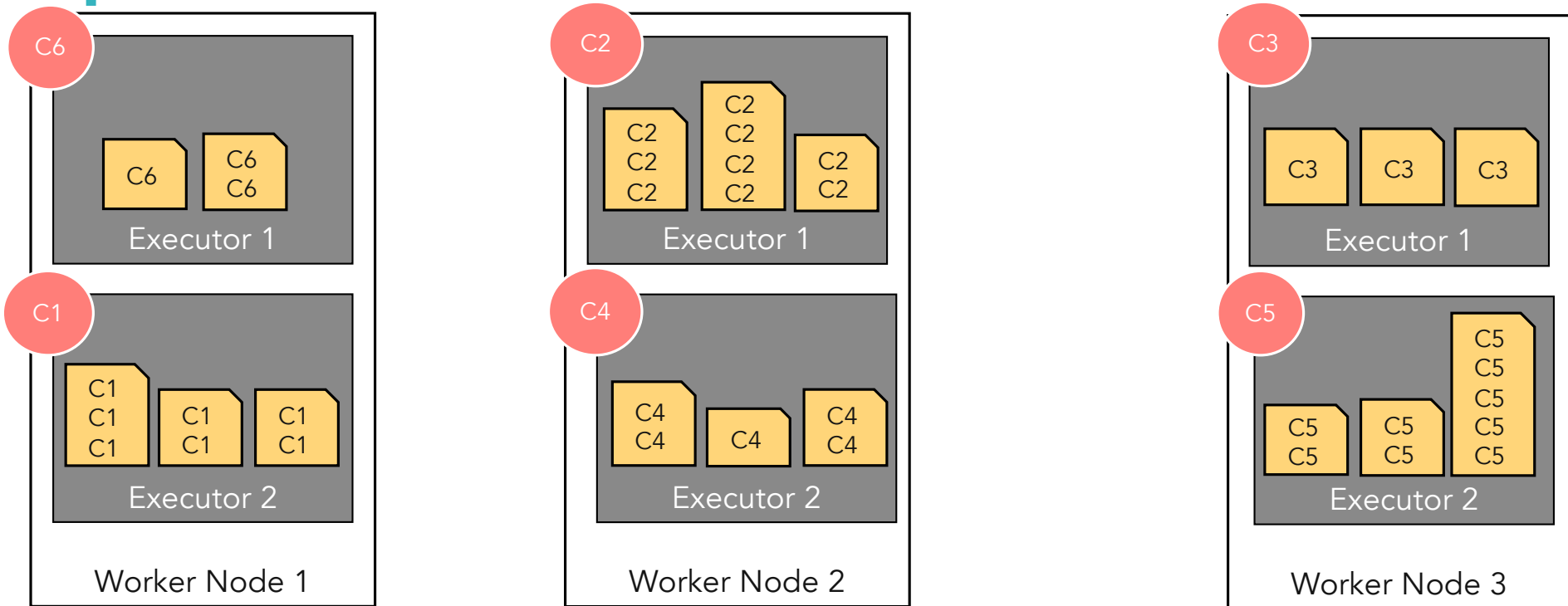
C1, C2, C3, C4, C5, C6

# Spark Shuffle



C1, C2, C3, C4, C5, C6

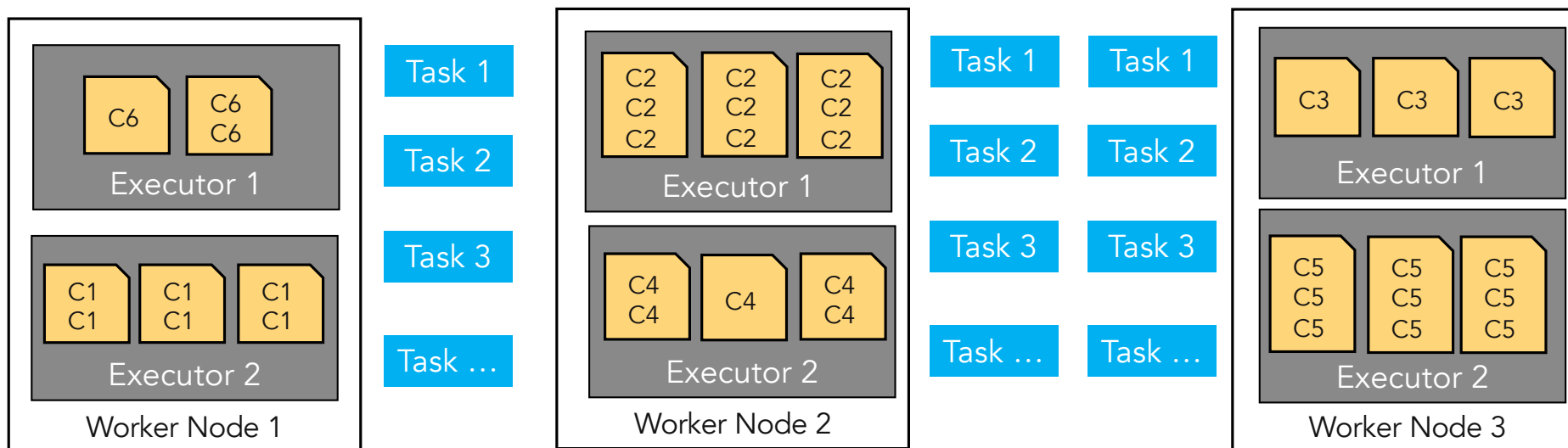
# Spark Shuffle



C1, C2, C3, C4, C5, C6

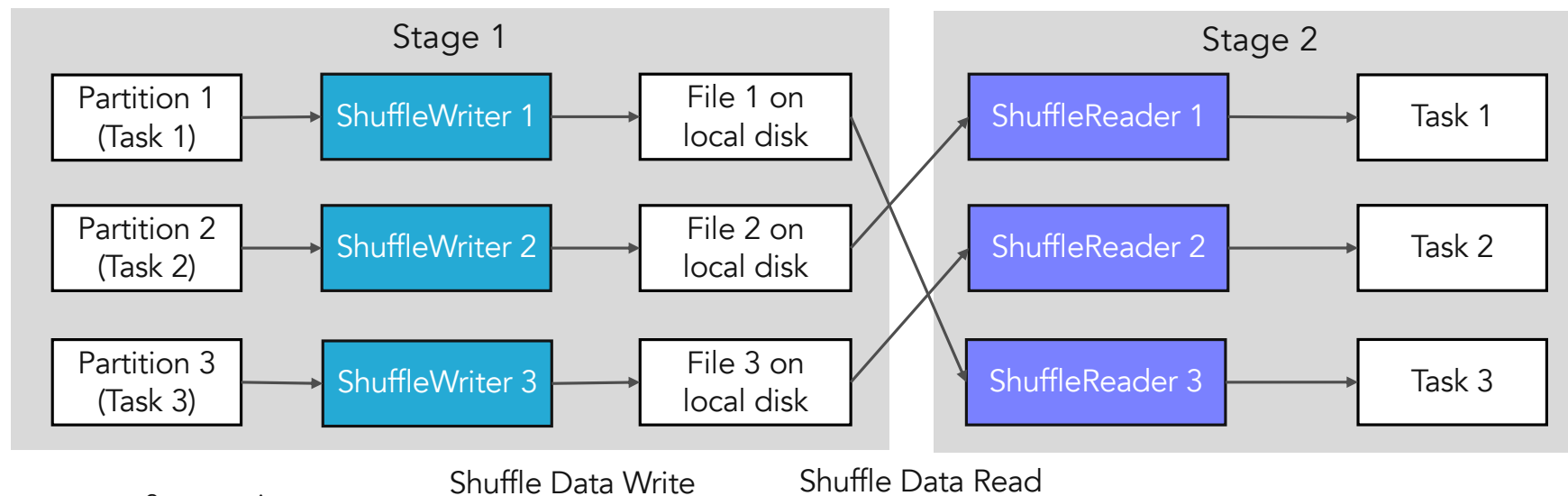
# Shuffle Partitions

- What are shuffle partitions?
- After shuffle process Spark creates multiple partitions and no. of tasks based on that.
- `spark.sql.shuffle.partitions`. Default value is 200.



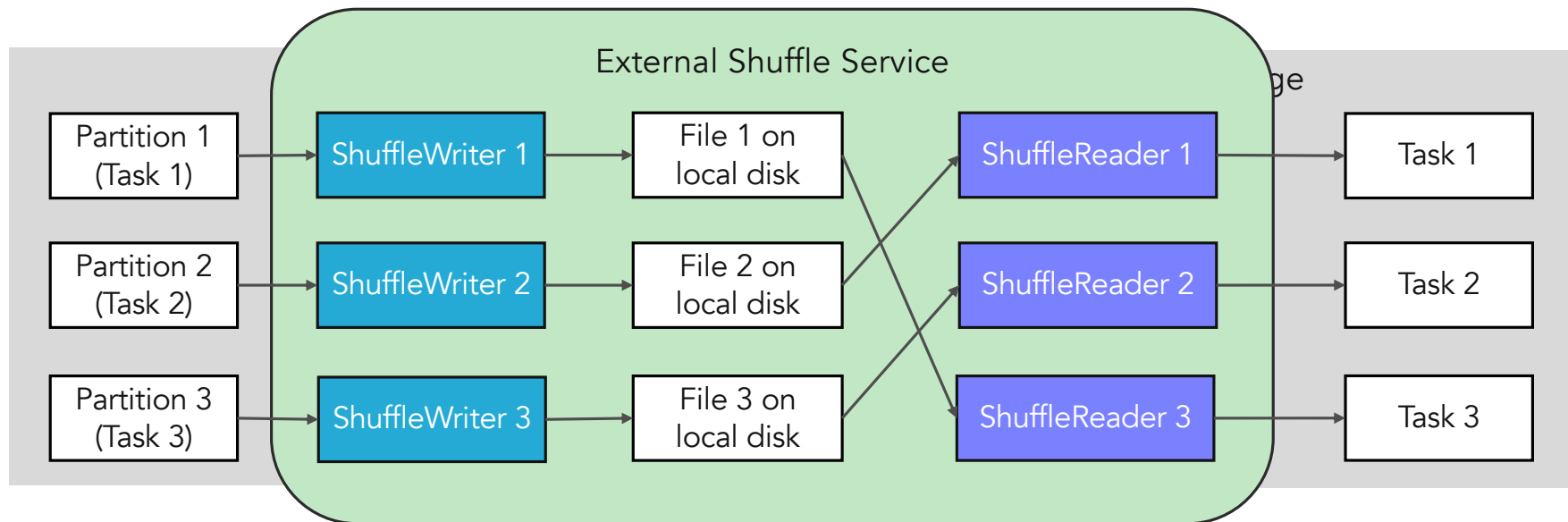
# External Shuffle Service

- Enabled using `spark.shuffle.service.enabled` (`spark-defaults.conf`)
- ShuffleWriter – Writes partitions to data files on disk, along with index files.
- `/tmp/spark-[app-id]/[shuffle-id]/[map-id]/`
- ShuffleReader – Reads data from data files on disk and provides it to the post-shuffle tasks.



# External Shuffle Service

- ESS = ShuffleWriter + ShuffleReader + Others
- Runs in its own JVM as a separate daemon process than executors and drivers.
- Even if executors die, the files will be available during restart and can be used.





# Shuffle Process

- Writes individual keys to files (IO).
- Moves the files with keys to their corresponding executor handler over the network.
- Does the aggregation in individual executors.
- Transformations that cause shuffle are called wide transformations.
- Shuffle service (another program/component) that writes data to files and then executors copy the records from this service.
- Shuffle creates stages.



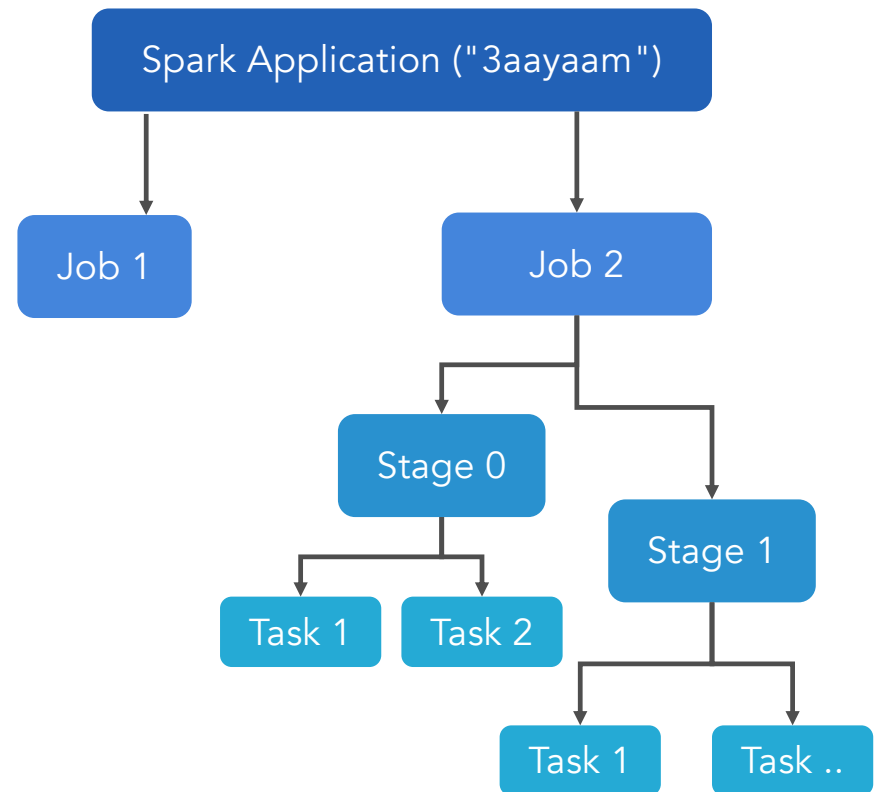
# Application & Job with SHUFFLE

```
... import SparkSession
... import SparkConf

spark = ...appName("3aayaam")...

df1 = ...
df2 = ...groupBy("id")
df3 = ...
df4 = ...

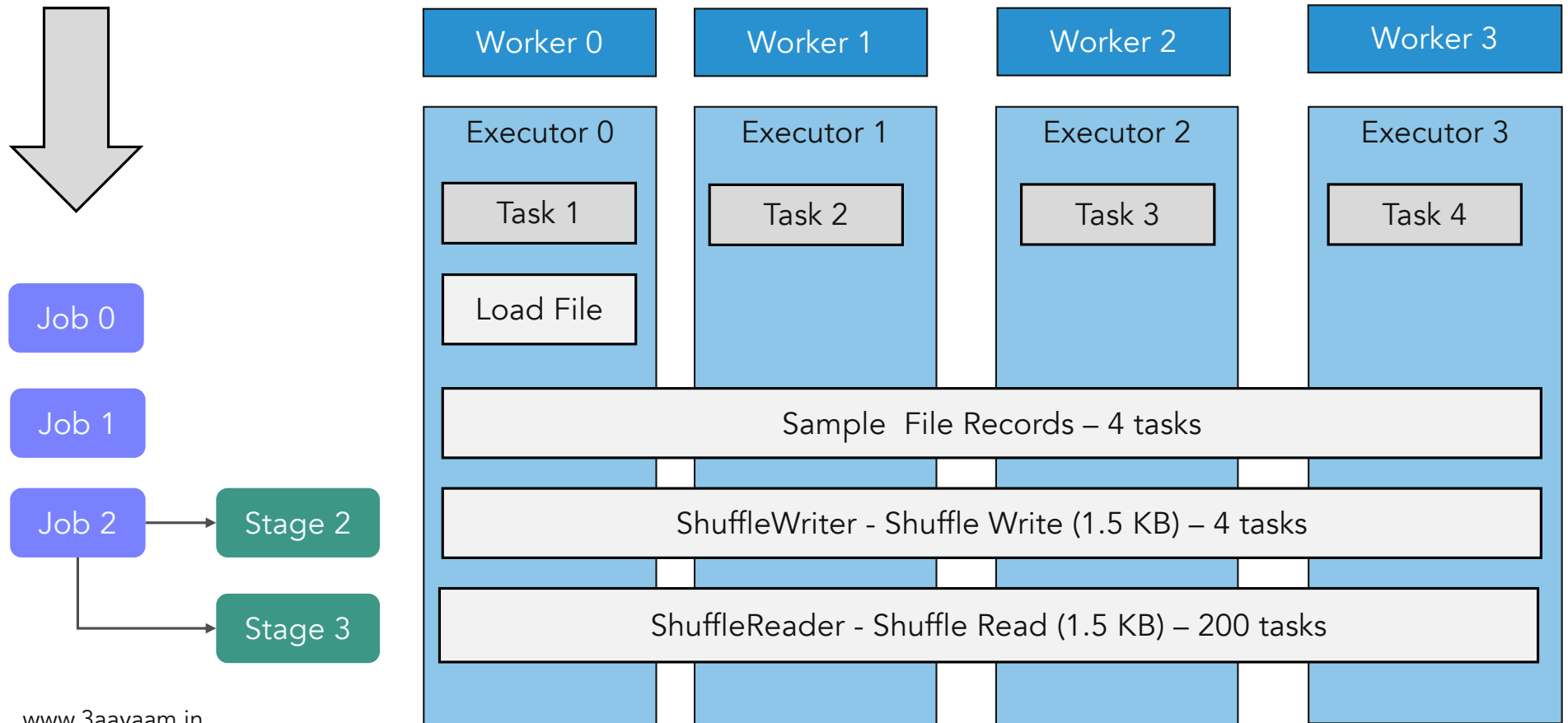
df4.write...save() ←
df2.write...save(...) ←
```



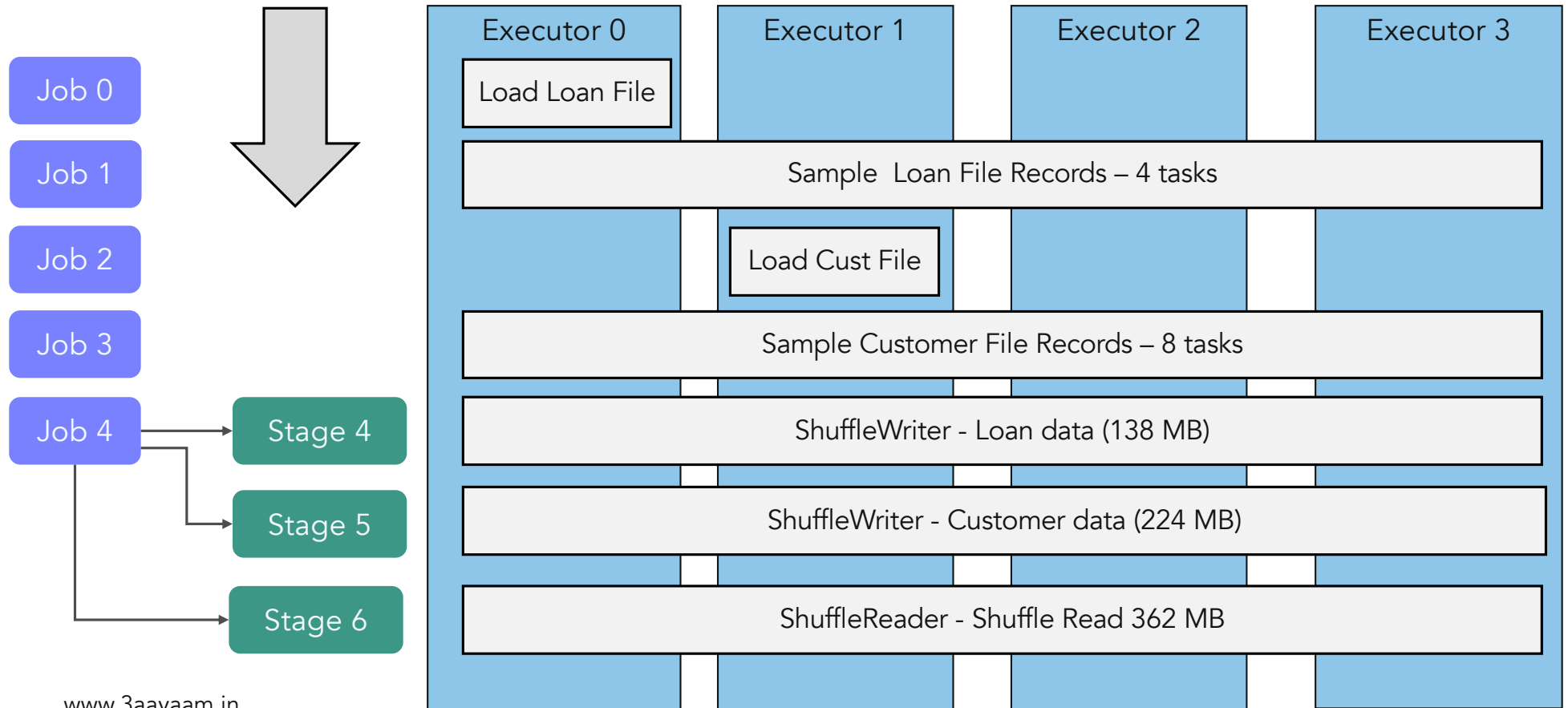
# Shuffle Process in Action

- Use WIDE transformations.
    - `groupBy`
    - `join`
  - Write to output file.
  - Spark UI
    - Explore Application, Jobs, Stages
    - DAG Visualization
    - SQL/Dataframe
  - Input Partitions (file read)
  - Shuffle input stage partitions (shuffle write)
  - Shuffle output stage partitions (shuffle read)
- `spark.conf.get('spark.sql.shuffle.partitions')`
  - `<dataframe>.rdd.getNumPartitions()`

# Shuffle in groupBy()

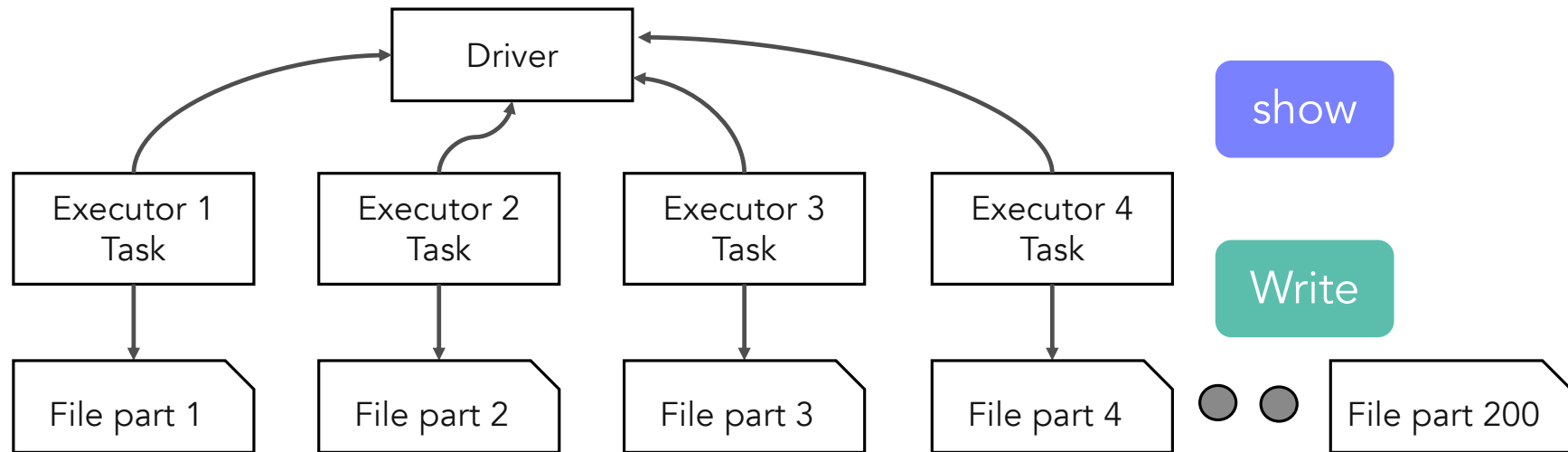


# Shuffle in join()



# Shuffle Process in Action

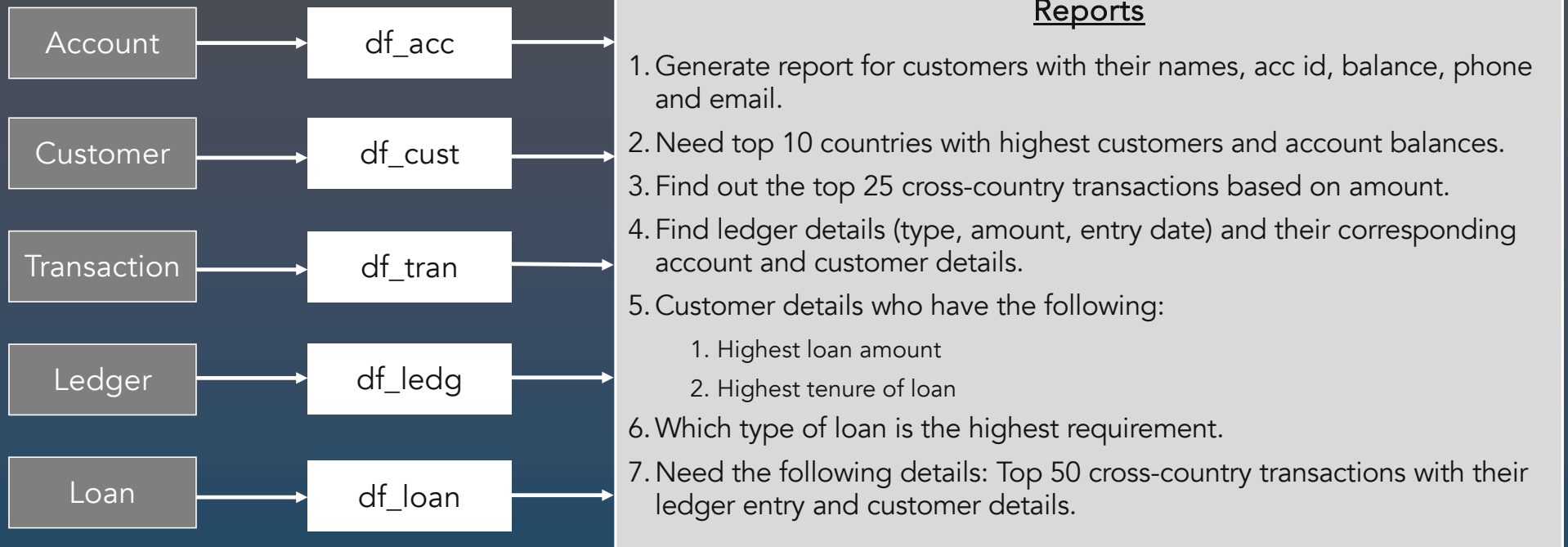
Always use DataFrame Writer to explore and analyze SHUFFLE process !!



# What should be Shuffle Partition no.?

- Input partitions – (File size/128mb) or (File size/4mb)
- Shuffle partitions
  - Default value of 200 is almost always non-optimal!!
  - `spark.sql.shuffle.partitions` is used to control the no. of partitions
- What should be the value of `spark.sql.shuffle.partitions`?
  - Post shuffle, individual partition size should be less than or equal to 200 mb.
  - No. of shuffle partitions (`spark.sql.shuffle.partitions`)  $\leq$  (Shuffle stage data size) / 200mb
  - No. of shuffle partitions  $\geq$  no. of cores in the cluster
- Output partitions
  - Repartition
  - Coalesce

# HandsOn – Project 2 (Finance)



- Explore Spark UI. Applications, Jobs, Stages, Shuffle Write & Read etc.



# Transformations Actions 1 - Summary

- df.columns
- df.distinct()
- df.filter()
- df.where()
- df.limit()
- df.printSchema()
- df.select()
- df.describe( [ 'col1', 'col2' ] )
- df.sort("col")
- df.drop()
- df.dropna()
- df.agg()
- df.groupBy()
- df.join()
- df.orderBy
- df.selectExpr()
- df.subtract()
- df.union()
- df.withColumn()
- df.show()
- df.count()
- df.write...save()
- df.first()
- df.head(n)
- df.tail(n)
- spark.read...load()

Assignment 1. Which are transformations and which are actions?  
Assignment 2. Which are Wide Transformations and which are Narrow Transformations?

# Summary

[www.3aayaam.in](http://www.3aayaam.in)

- Spark basic transformation
- One-by-one and chain transformation
- Actions and jobs
- Lazy evaluation
- Transformations, Actions
- “col” Function and its methods.
- Spark UI
- Shuffle or Wide transformations
- Shuffle process steps
- Application, Job, Stages



Thank You  
[www.3aayaam.in](http://www.3aayaam.in)